

[종합실습4] — 인터랙티브 할 일 관리 앱

요구사항 체크리스트

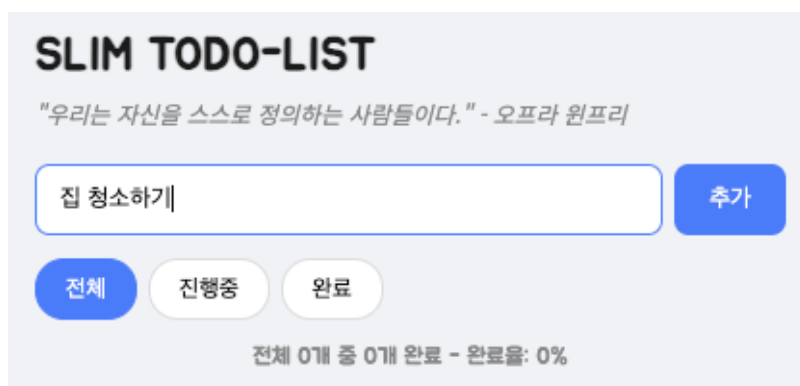
- ✓ ~~추가 버튼 또는 Enter 키로 할 일 추가 (반 값 방지)~~
- ✓ ~~체크박스로 완료 토글 (취소선), ✕ 버튼으로 삭제~~
- ✓ ~~전체 / 진행중 / 완료 필터 + 하단 요약 (전체 완료 개수)~~
- ✓ ~~이벤트 위임 (부모 나에 리스너 1회)으로 동적 항목 처리~~
- ✓ ~~storage.js / app.js 모듈 분리, type="module" 사용~~
- ✓ ~~localStorage로 저장 복원~~
- ✓ ~~fetch로 오늘의 한마디 로드 (실패 시 기본 문구)~~

1. 추가 (Enter)

| 입력 후 Enter → 목록에 추가되고 입력창이 비워짐

결과 화면

추가 전



추가 후(Enter 입력, 추가 버튼 클릭 동일)

SLIM TODO-LIST

"우리는 자신을 스스로 정의하는 사람들이다." - 오프라 윈프리

☐

전체 1개 중 0개 완료 - 완료율: 0%

관련 소스코드

app.js

```

56  function addTodo() {
57      const task = todoInput.value.trim();
58      //JS에선 0, "", null, undefined, NaN등은 전부 falsy 값.
59      //공백이나 이상한 값 들어오면 예외 처리.
60      if (!task) {
61          alert("올바른 값을 입력해주세요");
62          todoInput.value = "";
63          return;
64      }
65
66      todos.push({
67          id: Date.now(),
68          task: task,
69          done: false
70      });
71
72      todoInput.value = "";
73      saveTodos(todos);
74      render();
75  }
76
77  function addByEnter(e){
78      if (e.key === "Enter") addTodo();
79  }
80
81  addBtn.addEventListener("click", addTodo);
82  todoInput.addEventListener("keyup", addByEnter); //keydown을 사용하니 한글에서 두번씩 추가되는 오류가 발생함.

```

2. 완료 토글

체크박스 클릭 → 취소선 표시, 하단 완료 개수 +1

결과 화면

SLIM TODO-LIST

"당신이 할 수 있다고 생각하든 할 수 없다고 생각하든, 당신이 옳다." - 헨리 포드

추가전체진행중완료☒ 잡 청소하기X☐ 강아지 밥주기X☒ 운동하기X☐ Study Java/SpringX

전체 4개 중 2개 완료 - 완료율: 50%

관련 소스코드

app.js (101라인~)

```
84  const todoList = document.querySelector(".list ul");
85  todoList.addEventListener("click", function(e) {
86    const li = e.target.closest("li");
87    if (!li) return;
88    const id = Number(li.getAttribute("data-id"));
89
90    //삭제 버튼 클릭 로직
91    if (e.target.textContent === "X"){
92      //confirm은 브라우저 내장함수, 취소 - false, 확인 - true 반환.
93      if(!confirm("정말 삭제하시겠습니까?")) return;
94      todos = todos.filter(function (t) {
95        return t.id !== id;
96      });
97      saveTodos(todos);
98      render();
99    }
100
101    //체크박스 클릭 토글 로직
102    if ((e.target.type === "checkbox" || e.target.tagName === "SPAN" || e.target.tagName === "LI")){
103      const todo = todos.find(function (t) {
104        return t.id === id;
105      });
106      if(todo) todo.done = !todo.done;
107      saveTodos(todos);
108      render();
109    }
110  });
```

3. 필터

| '완료' 필터 클릭 → 완료 항목만 표시

결과 화면

[전체 필터 캡처]

SLIM TODO-LIST

"당신이 할 수 있다고 생각하든 할 수 없다고 생각하든, 당신이 옳다." - 헨리 포드

추가

전체 진행중 완료

☒ 집 청소하기

X

☐ 강아지 밥주기

X

☒ 운동하기

X

☐ Study Java/Spring

X

전체 4개 중 2개 완료 - 완료율: 50%

[완료]

SLIM TODO-LIST

"당신이 할 수 있다고 생각하든 할 수 없다고 생각하든, 당신이 옳다." - 헨리 포드

추가

전체 진행중 완료

☒ 집 청소하기

X

☒ 운동하기

X

전체 4개 중 2개 완료 - 완료율: 50%

[진행중]

SLIM TODO-LIST

"당신이 할 수 있다고 생각하든 할 수 없다고 생각하든, 당신이 옳다." - 헨리 포드

추가

전체 진행중 완료

☐ 강아지 밥주기

X

☐ Study Java/Spring

X

전체 4개 중 2개 완료 - 완료율: 50%

관련 소스코드

app.js

필터 이벤트 로직

```
114 const filters = document.querySelector(".filters");
115 filters.addEventListener("click", function (e){
116     //filters 클래스의 버튼들에 미리 data-filter 속성(all, processing, done)을 넣어워서 dataset으로 꺼내서 쓸 수 있다.
117     if(!e.target.dataset.filter) return;
118
119     //필터 버튼 전부 돌면서 active 설정된 클래스 삭제 - 버튼 스타일링을 위해.
120     document.querySelectorAll(".filters button").forEach(function (btn) {
121         btn.classList.remove("active");
122     });
123     //현재 선택된 버튼에 active 클래스 할당
124     e.target.classList.add("active");
125
126     currentFilter = e.target.dataset.filter;
127     render();
128 });
```

render시 필터로직을 통해 결정된 currentFilter로 목록에 보여줄 값 선택.

```

13 //필터 버튼 적용 - .filters 이벤트를 통해 currentFilter 변경
14 let filtered = todos;
15 if (currentFilter === "processing"){
16     filtered = todos.filter(function (t) {
17         return !t.done;
18     });
19 } else if(currentFilter === "done"){
20     filtered = todos.filter(function (t){
21         return t.done;
22     });
23 }
24
25 //todos 중 data-filter 조건을 만족하는 것들만 filtered 배열에 걸림.
26 filtered.forEach(function(t){
27     const li = document.createElement("li");
28     li.setAttribute("data-id",t.id); //data-id="todo.id"
29     if (t.done) li.classList.add("done"); //취소선 스타일 적용을 위해 li태그마다 done 클래스를 넣어줌.
30
31     const checkBox = document.createElement("input");
32     checkBox.type = "checkbox";
33     checkBox.checked = t.done;
34 }

```

4. 삭제 / 영속성

×로 항목 삭제, 항목 추가 후 새로고침 → localStorage로 유지

결과 화면

삭제 전

SLIM TODO-LIST

"행복하기 위해선 절대 다른 사람들을 너무 의식해서는 안 된다." - 알베르 카뮈

☒
집 청소하기
×

☐
강아지 밥주기
×

☒
운동하기
×

☐
Study Java/Spring
×

☐
세차하기
×

전체 5개 중 2개 완료 - 완료율: 40%

삭제 후

SLIM TODO-LIST

"행복하기 위해선 절대 다른 사람들을 너무 의식해서는 안 된다." - 알베르 카뮈

추가

전체

진행중

완료

☒ 집 청소하기

X

☐ 강아지 밥주기

X

☒ 운동하기

X

☐ Study Java/Spring

X

전체 4개 중 2개 완료 - 완료율: 50%

새로고침 후 유지 - 명언은 바뀜

SLIM TODO-LIST

"불행의 원인은 늘 자신에게 있다." - 블레즈 파스칼

추가

전체

진행중

완료

☒ 집 청소하기

X

☐ 강아지 밥주기

X

☒ 운동하기

X

☐ Study Java/Spring

X

전체 4개 중 2개 완료 - 완료율: 50%

관련 소스코드

storage.js

```

1  export function loadTodos() {
2    const data = localStorage.getItem("todos");
3    return data ? JSON.parse(data) : [];
4  }
5
6  export function saveTodos(todos){
7    localStorage.setItem("todos", JSON.stringify(todos));
8  }

```

app.js [삭제 + saveTodos 호출]

```

86  const todoList = document.querySelector(".list ul");
87  todoList.addEventListener("click", function(e) {
88    const li = e.target.closest("li");
89    if (!li) return;
90    const id = Number(li.getAttribute("data-id"));
91
92    //삭제 버튼 클릭 로직
93    if (e.target.textContent === "X"){
94      //confirm은 브라우저 내장함수, 취소 - false, 확인 - true 반환.
95      if(!confirm("정말 삭제하시겠습니까?")) return;
96      todos = todos.filter(function (t) {
97        return t.id !== id;
98      });
99      saveTodos(todos);
100     render();
101   }

```

5. 비동기 (오늘의 한마디)

| 상단 오늘의 한마디 로드 (인터넷 차단 시 기본 문구로 대체)

결과 화면

[정상 로드]

SLIM TODO-LIST

"삶도 모르는데 어찌 죽음을 알겠는가?" - 공자

[오프라인 시 기본 문구]

SLIM TODO-LIST

안되면 될 때 까지 (기본 문구로 대체 되었습니다.)

추가

관련 소스코드

app.js

```
135 async function loadQuote() {
136     const todayWord = document.getElementById("todayWord");
137
138     try {
139         //fetch에서 예외 터지면 기본문구
140         const res = await fetch("https://korean-advice-open-api.vercel.app/api/advice");
141         const data = await res.json();
142         todayWord.textContent = "\"" + data.message + "\" - " + data.author;
143     } catch (err){
144         todayWord.textContent = "안되면 될 때 까지 (기본 문구로 대체 되었습니다.)";
145     }
146 }
147
148 loadQuote();
```

6. 이벤트 위임

| UI에 리스너 1개만 걸고 체크박스/삭제 버튼 로직 구분

관련 소스코드

app.js

```

86 const todoList = document.querySelector(".list ul");
87 todoList.addEventListener("click", function(e) {
88     const li = e.target.closest("li");
89     if (!li) return;
90     const id = Number(li.getAttribute("data-id"));
91
92     //삭제 버튼 클릭 로직
93     if (e.target.textContent === "X"){
94         //confirm은 브라우저 내장함수, 취소 - false, 확인 - true 반환.
95         if(!confirm("정말 삭제하시겠습니까?")) return;
96         todos = todos.filter(function (t) {
97             return t.id !== id;
98         });
99         saveTodos(todos);
100         render();
101     }
102
103     //체크박스 클릭 토글 로직
104     if ((e.target.type === "checkbox" || e.target.tagName === "SPAN" || e.target.tagName === "LI")){
105         const todo = todos.find(function (t) {
106             return t.id === id;
107         });
108         if(todo) todo.done = !todo.done;
109         saveTodos(todos);
110         render();
111     }
112 });

```

종합 평가 (+ 추가과제 명시)

이번 과제 CSS(대부분 AI 코드 사용)는 간단히 적용하고, JS 이벤트와 함수들 적용에 시간을 많이 썼다. 이벤트 위임과 async & await, localStorage, 스크립트 모듈화 등 많은 내용이 있어서 단 시간에 완벽히 이해하진 못한 것 같지만, 핵심적인 내용은 어느정도 이해한 것 같다. JS 부분이라 간단하지만 기능적으로 구현해보고 싶은 부분이 몇 개 있어서 추가적인 부분을 많이 했다.

html 속성 값들을 변수처럼 넣었다 뺐다 하면서 렌더링 시키는 목록을 바꾸거나, 스타일을 바꾸는 부분이 이전에 Spring 공부하면서 DB 값을 수정하면 뷰가 따라오는 것과 비슷한 구조라 이해하기 수월했다. 그리고 CSS를 적용하면서도 이전 배웠던 hover시 transform을 준다던가 하는 부분들을 적용하는것도 재밌었다.

아직 JS 문법에는 익숙하지 않아서 falsy 값으로 조건 체크나 !를 통해 boolean 반전? 같은 부분은 어색하고 쓸때마다 찾아봐야 한다. 그래도 Java에 비해 JS는 덜 strict한 언어인 거 같아서 편하긴하다.

나의 평가: CSS에 많은 시간을 쏟고 싶지않아서 큰 틀은 AI코드를 사용한 점, 한글로 할 일 추가 시 공백 값이 한번 더 입력(이중 입력)돼서 `alert("올바른 값을 입력해주세요");` 이 뜨는 버그? 를 이해안하고 적용하고 싶진 않아서 일단 적용 안했다. 이런점들을 보완하면 더 보기 좋은 앱 페이지로 만들 수 있을 것 같다.

추가과제

- **삭제 시 confirm 확인창** — 삭제 버튼 클릭 시 `confirm()` 으로 사용자에게 재확인, 취소 시 삭제 중단
- **li 클릭으로 체크박스 토글** — 체크박스 뿐 아니라 텍스트(span)나 li 자체 클릭으로도 완료 토글 가능하도록 이벤트 위임 조건 확장 - 삭제 로직이 위에 체크박스 클릭 로직보다 위에 있어야 먼저 작동하여, 삭제 버튼 클릭 시 삭제 로직이 먼저 발동 됨.
- **필터 버튼 active 스타일** — 현재 선택된 필터 버튼에 `.active` 클래스를 동적으로 부여해서 시각적으로 어떤 필터가 선택됐는지 표시
- **빈 값 입력 시 alert 안내** — 공백만 입력하거나 빈 값일 때 `alert()` 로 사용자에게 안내 메시지 표시 후 입력창 초기화
- **완료율 표시** — 삼항연산자로 0으로 나누기 방지 및 완료율(%)을 계산해서 표시